School of Computer Science

COMP 8347 “Internet Applications and Distributed Systems”

Summer 2026

Week 2
Multi-tier Architecture; Django Introduction (MVT Pattern).

DR. OLENA SYROTKINA

University of Windsor

Copyright Notice

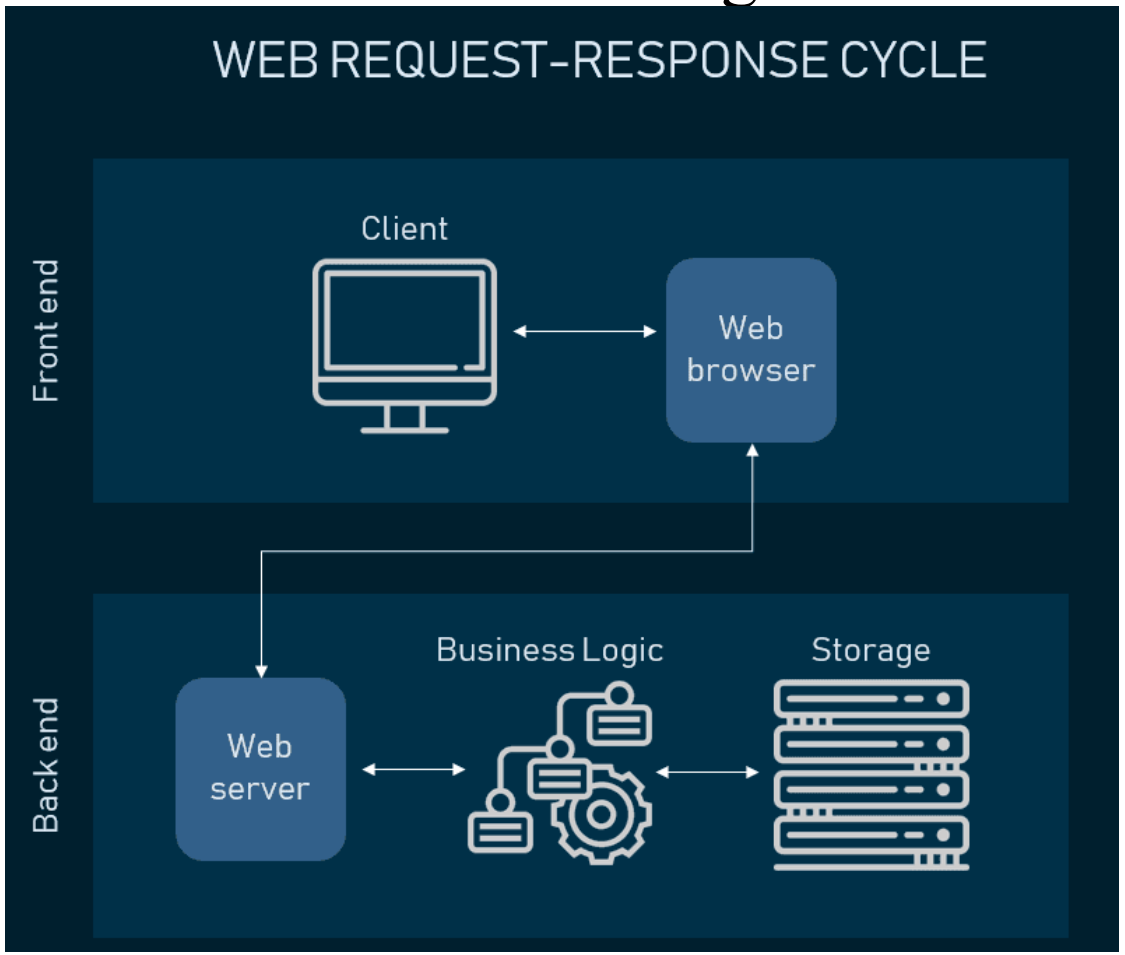
This content is protected and may not be shared, uploaded, or distributed. Any unauthorized distribution, reproduction or sharing of this material, including uploading to **CourseHero, Chegg, StuDocu** or other websites, is **strictly prohibited** and may be subject to legal action. Students are granted access to this material solely for their personal use and may not use it for any other purpose without the express written consent of the instructor. Thank you for respecting our intellectual property rights.

Agenda

- Introduction
- Multi-tier Architecture: The Concept
- Django Framework
- Setting Up Django
- Your First Django App

Introduction

How does Netflix/Instagram/Amazon actually work behind the scenes?



Think about this…..

1. Why not just write everything in one file? What problems would arise?

2. If our app had 1 million users tomorrow, what would break first?

client/browser → web server → business logic → storage (database).

Introduction

Bad example: Single-file “all-in-one” Python web app

```python
# big_app.py  (one giant file)

import sqlite3
from http.server import BaseHTTPRequestHandler, HTTPServer

DB_PATH = "notes.db"

# ---- DATABASE CREATION AND LOGIC ----
conn = sqlite3.connect(DB_PATH)
c = conn.cursor()
c.execute("""CREATE TABLE IF NOT EXISTS notes (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                text TEXT
            )""")
conn.commit()

# ---- SERVER + HTML UI + BUSINESS LOGIC ----
class MyHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/":
            # show all notes and a form
            c = conn.cursor()
            notes = c.execute("SELECT text FROM notes").fetchall()
            page = "<h1>Notes</h1>"
            for n in notes:
                page += f"<p>{n[0]}</p>"
            page += """
                <form method="POST">
                    <input name="text">
                    <button>Add</button>
                </form>
            """
            self.send_response(200)
            self.end_headers()
            self.wfile.write(page.encode())
```

```python
    def do_POST(self):
        length = int(self.headers["Content-Length"])
        data = self.rfile.read(length).decode()
        note = data.split("=")[1]
        c = conn.cursor()
        c.execute("INSERT INTO notes (text) VALUES (?)", (note,))
        conn.commit()
        self.send_response(303)
        self.send_header("Location", "/")
        self.end_headers()

# ---- RUN SERVER ----
server = HTTPServer(("0.0.0.0", 8000), MyHandler)
print("Serving on http://localhost:8000")
server.serve_forever()
```

Introduction

1. Why not just write everything in one file? What problems arise (scalability, maintainability, security)?

1. Hard to grow (scalability)

• One file can run on only one computer at a time.

• If lots of people use the app, you can't easily split the work across many servers.

2. Hard to fix or change (maintainability)

•All code is mixed together – design, logic, database calls.

•A small change might break something far away in the file.

•New developers will have a hard time understanding it.

3. Less safe (security)

•No clear walls between parts, so a bug in one place can expose data.

•Harder to protect the database or hide secrets.

4. Other headaches

•One crash stops the whole app.

•Testing and updating take longer.

Introduction

2. If our app suddenly had 1 million users tomorrow, what would break first?

1. Most likely the database
•Too many people asking for data at the same time.
•It can run out of connections or slow down.

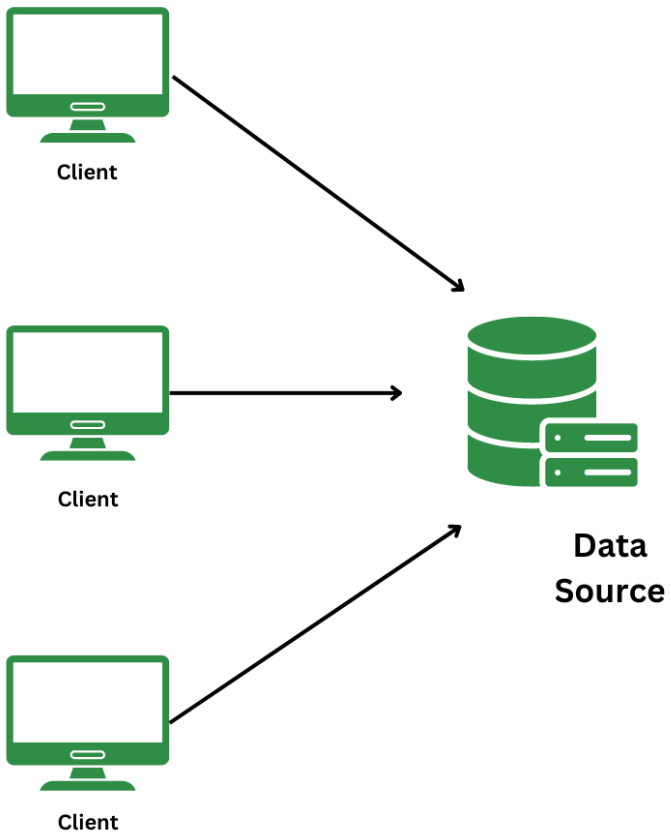
2. Then the web server/app code
•Too many requests waiting in line.
•Pages start loading slowly or time out.

3. Other possible weak spots
•Static files (images, CSS) if you don't use a content-delivery network (CDN).
•Any third-party services (email, payments) that have limits.

4. Simple fixes
•Add a cache or CDN so not every request hits the database.
•Run several copies of the app so more servers share the load.
•Use queues for slow tasks like sending emails.

Multi-tier Architecture: The Concept

Two-Tier Architecture



Microsoft Access Applications

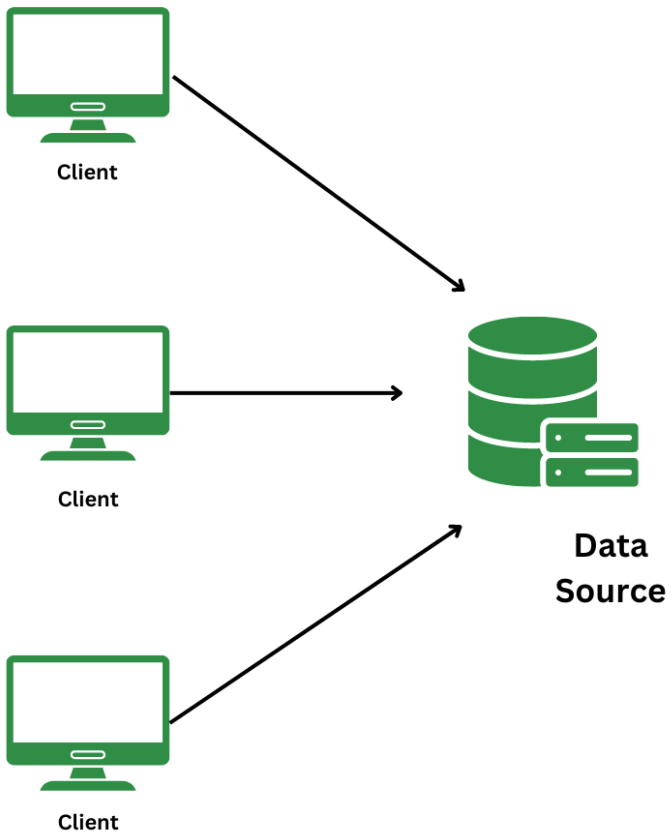
- **Concept:** An Access "front-end" file (.accdb) contains the forms, reports, and VBA logic.
- The back end is a shared Access or SQL Server database.
- **Why two-tier:** The Access front end connects *directly* to the database; all business logic lives partly in VBA and partly in stored procedures.

Desktop Inventory or POS Systems (Small Retail)

- Think of **small shops using a single Windows app** built with Visual Basic or Delphi.
- The program opens a direct ODBC connection to a local or shared MySQL/SQL Server database.
- No separate application server – logic for pricing, taxes, and reporting lives in the desktop executable.

Multi-tier Architecture: The Concept

Two-Tier Architecture



Early Client/Server Tools (Oracle Forms / PowerBuilder)

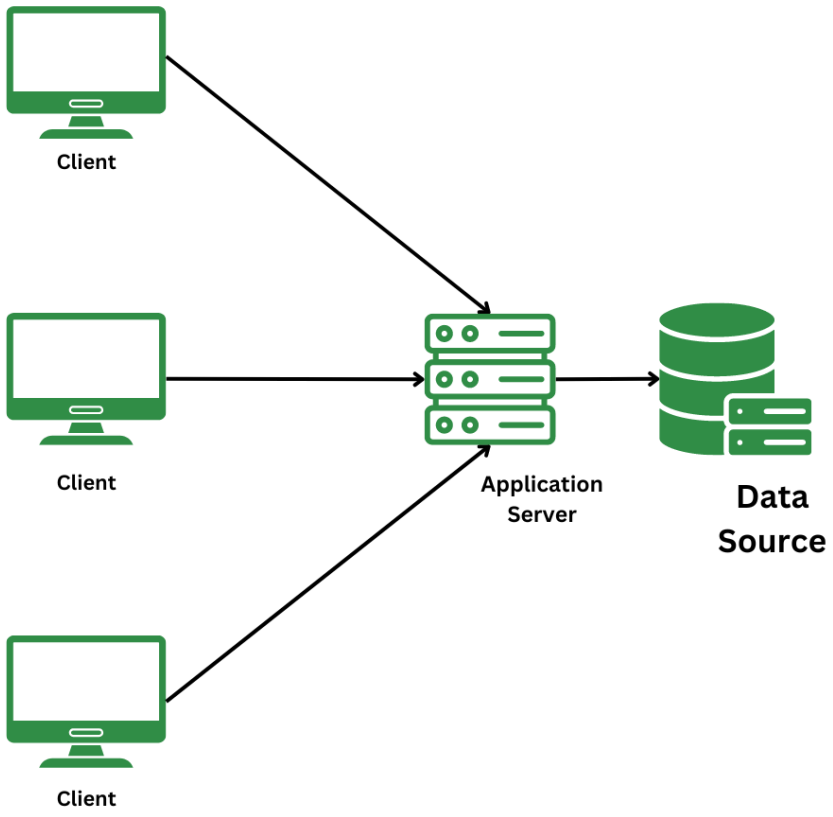
- 1990s corporate apps built with **Oracle Forms** or **Sybase PowerBuilder**.
- The rich desktop client spoke SQL directly to Oracle/Sybase databases.
- All rules were split between the client code and database stored procedures.

Simple Internal Line-of-Business Apps

- Example: A company's HR timesheet tool built in **Java Swing or .NET WinForms**, where the app connects straight to an internal SQL Server.
- Still common in factories, labs, and small offices where everyone's on the same LAN.

Multi-tier Architecture: The Concept

Three-Tier Architecture



1. Presentation Tier:

- What the user sees: HTML, CSS, JavaScript, templates.
- In Django: handled by **Templates.**
- Example: Login page with text boxes and a "Login" button.

2. Logic (Application) Tier:

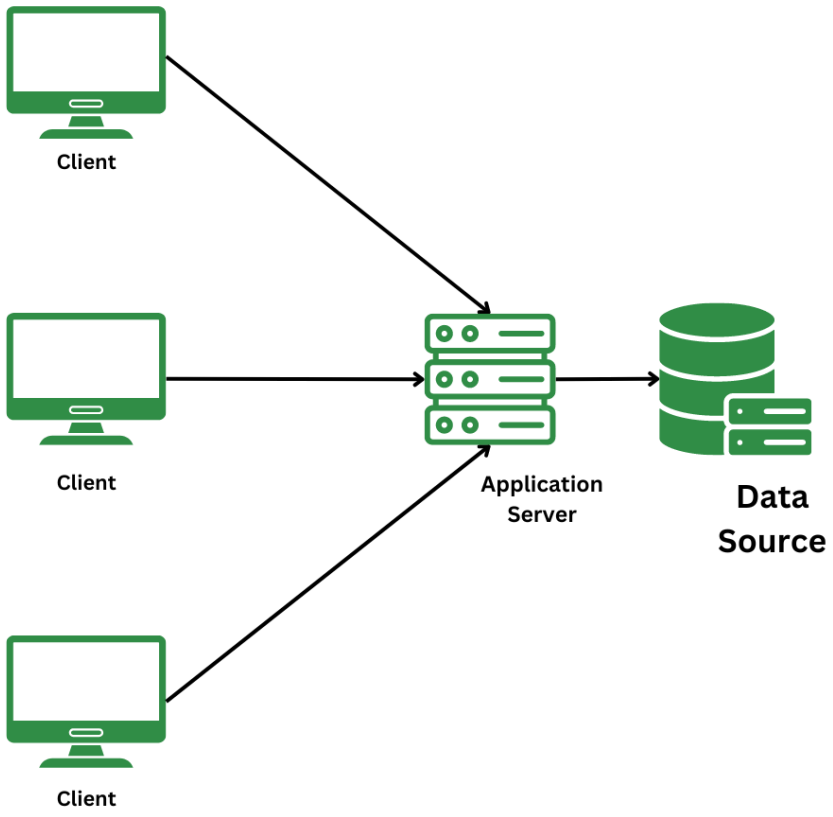
- Where rules live. "If username + password match, allow login."
- In Django: handled by **Views** (functions/classes).
- Example: Process login request, check database, return appropriate response.

3. Data Tier:

- Where information is stored.
- Databases such as SQLite, PostgreSQL, MySQL.
- In Django: handled by **Models** via the ORM.
- Example: Table Users stores name, password hash, email.

Multi-tier Architecture: The Concept

Three-Tier Architecture



Class Activity

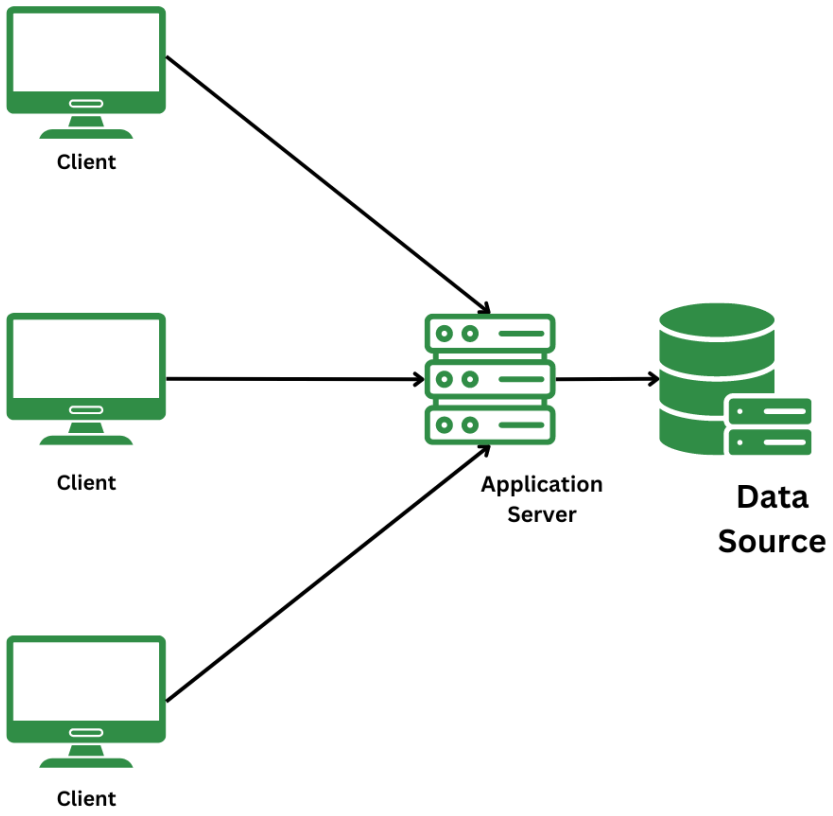
In pairs, pick an app you use daily (Instagram, Uber Taxi, Amazon Shopping, WhatsApp, etc.).

Write down:

- ✓ Presentation examples
- ✓ Logic examples
- ✓ Data examples

Multi-tier Architecture: The Concept

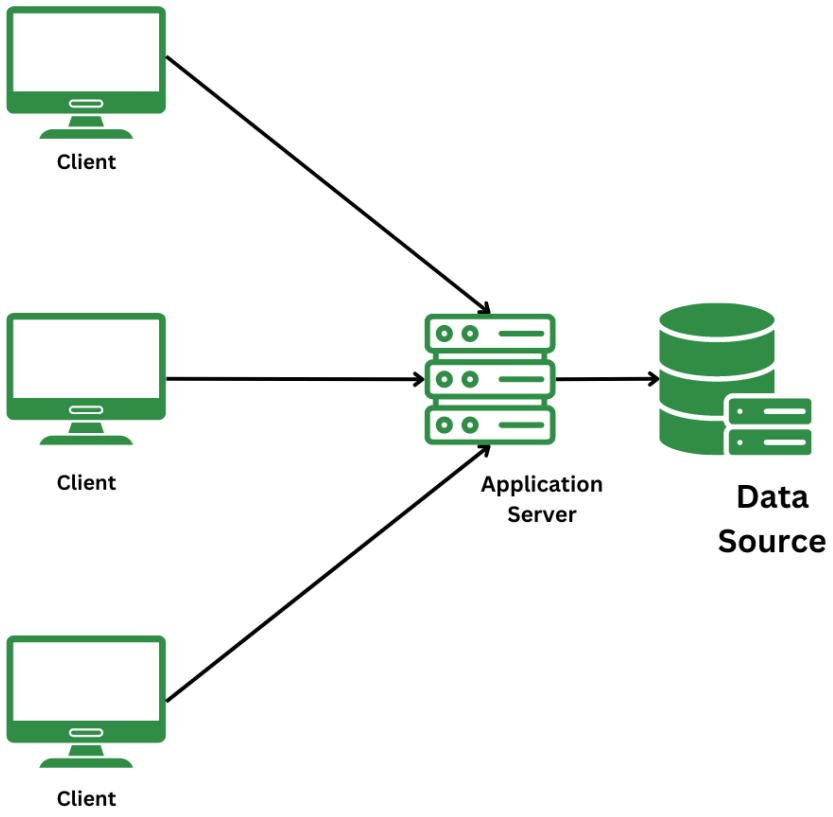
Three-Tier Architecture



1. E-Commerce Websites (e.g., Shopify, Etsy, Amazon stores)
Presentation tier: Web browser or mobile app using HTML/CSS/JavaScript or React.
Logic tier: Application servers running Ruby on Rails, Django, or Node.js.
Data tier: Relational database (e.g. PostgreSQL/MySQL) for orders, plus key-value stores (Redis) for caching.
The shop's business rules (pricing, discounts, payment workflows) live entirely in the middle tier.

Multi-tier Architecture: The Concept

Three-Tier Architecture



2. Online Banking Portals

Presentation tier: Customer browser or native banking app.

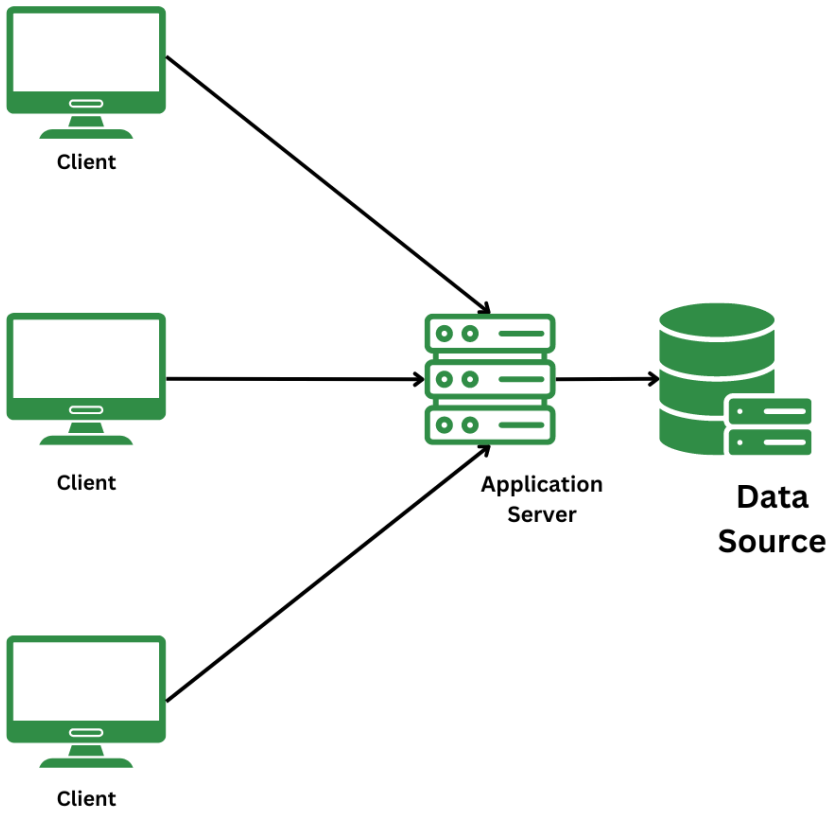
Logic tier: Java/Spring or .NET application servers with strict business rules and security checks.

Data tier: Core banking databases (Oracle/DB2) storing accounts and transactions.

High security and clear separation let banks scale front ends and enforce audit trails.

Multi-tier Architecture: The Concept

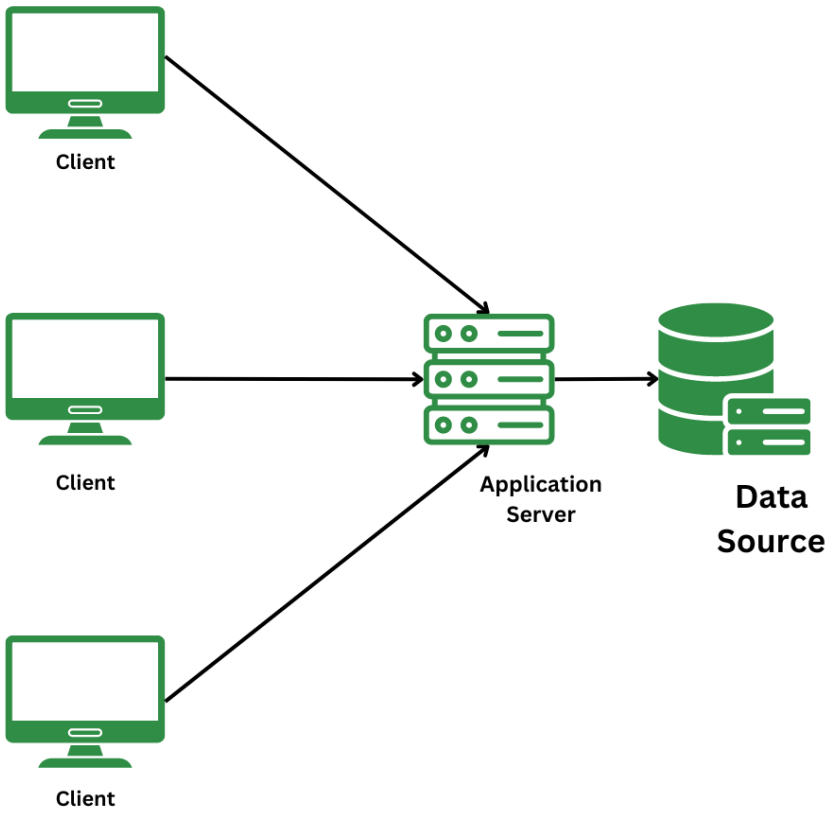
Three-Tier Architecture



3. Learning Management Systems (e.g., Brightspace, Moodle, Canvas, etc.)
Presentation tier: HTML/CSS/JavaScript front end or mobile app.
Logic tier: PHP, Python, or Java back-end servers handle enrollment, grading logic, and notifications.
Data tier: MySQL/PostgreSQL database holds courses, user records, and grades.

Multi-tier Architecture: The Concept

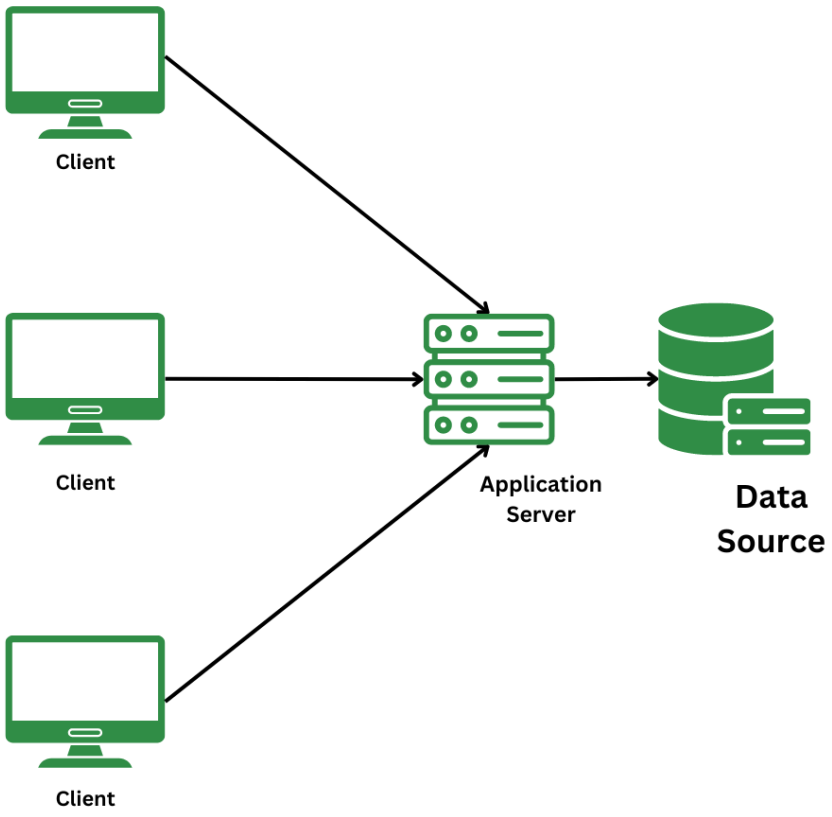
Three-Tier Architecture



4. Streaming Services (e.g., Netflix Web App)
Presentation tier: React/JavaScript UI running in browsers, TV apps, or mobile clients.
Logic tier: Microservices cluster (Java, Node.js) serving APIs for catalog, recommendations, playback sessions.
Data tier: Distributed databases for video metadata, user profiles, and viewing history.

Multi-tier Architecture: The Concept

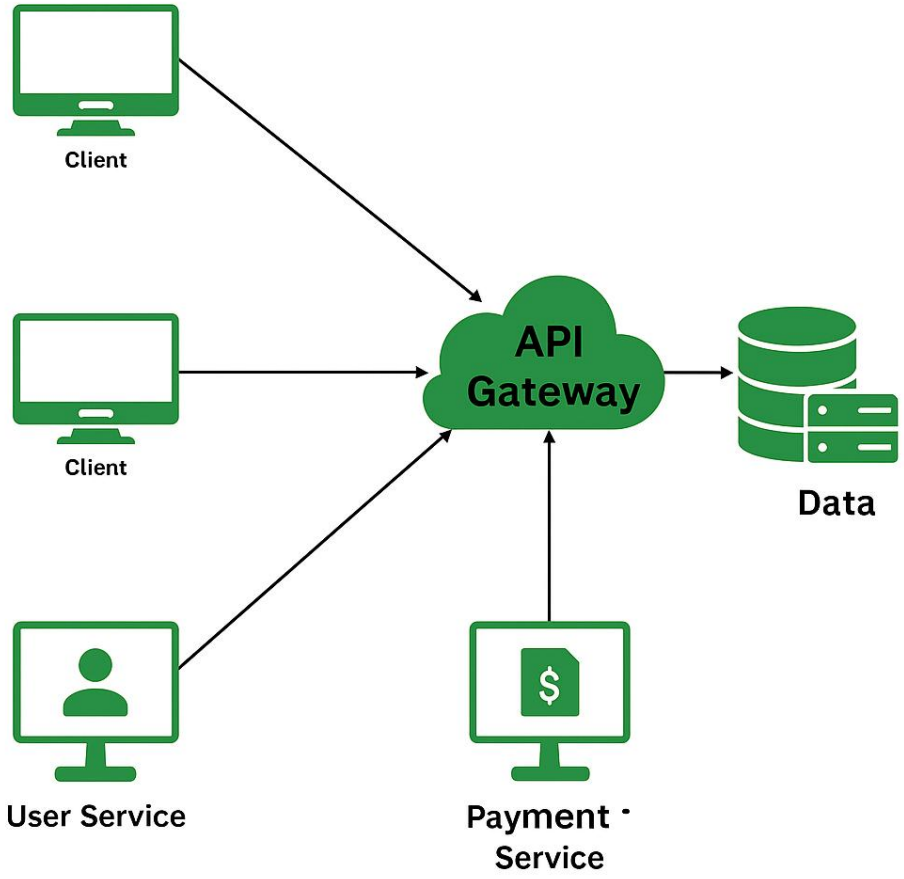
Three-Tier Architecture



5. Typical Corporate Intranet Portals
Presentation tier: Browser-based front end built with Angular or plain templates.
Logic tier: Middleware (Django, Flask, or .NET) enforcing company workflows (leave requests, expense approvals).
Data tier: Central database storing HR and finance records.

Multi-tier Architecture: The Concept

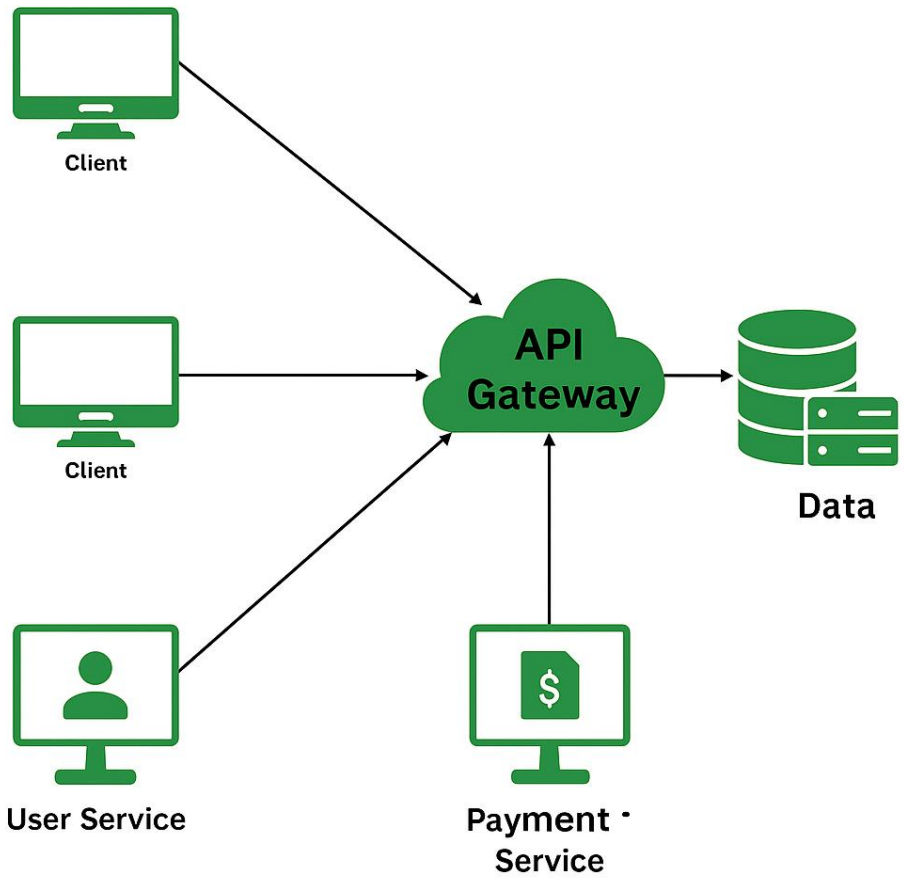
N-tier / Microservices:



Clients: End-user devices (browsers, mobile apps) sending requests.
API Gateway: A single entry point that routes client requests to the correct microservice. *Handles authentication, load balancing, rate limiting, and aggregates responses.*
User Service: Independent microservice responsible for user-related functions such as login, profile updates, etc.
Payment Service: Separate microservice dedicated to payment processing and billing logic.
Data Layer: Databases or storage systems where persistent data lives.

Multi-tier Architecture: The Concept

N-tier / Microservices:

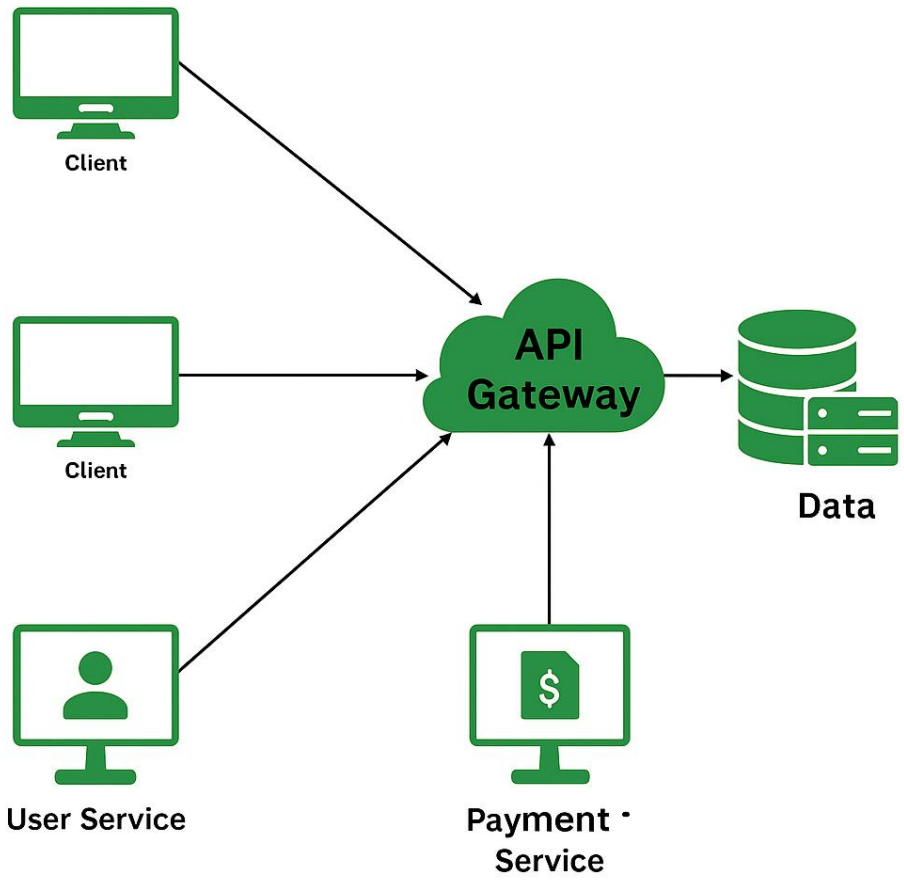


How does it work?

Clients send HTTP/REST (or GraphQL) requests to the **API Gateway**.
The **API Gateway** decides which microservice (User or Payment) should handle each request.
Each microservice can query or update the **Data** store as needed.
The gateway then returns the combined response to the client.

Multi-tier Architecture: The Concept

N-tier / Microservices:

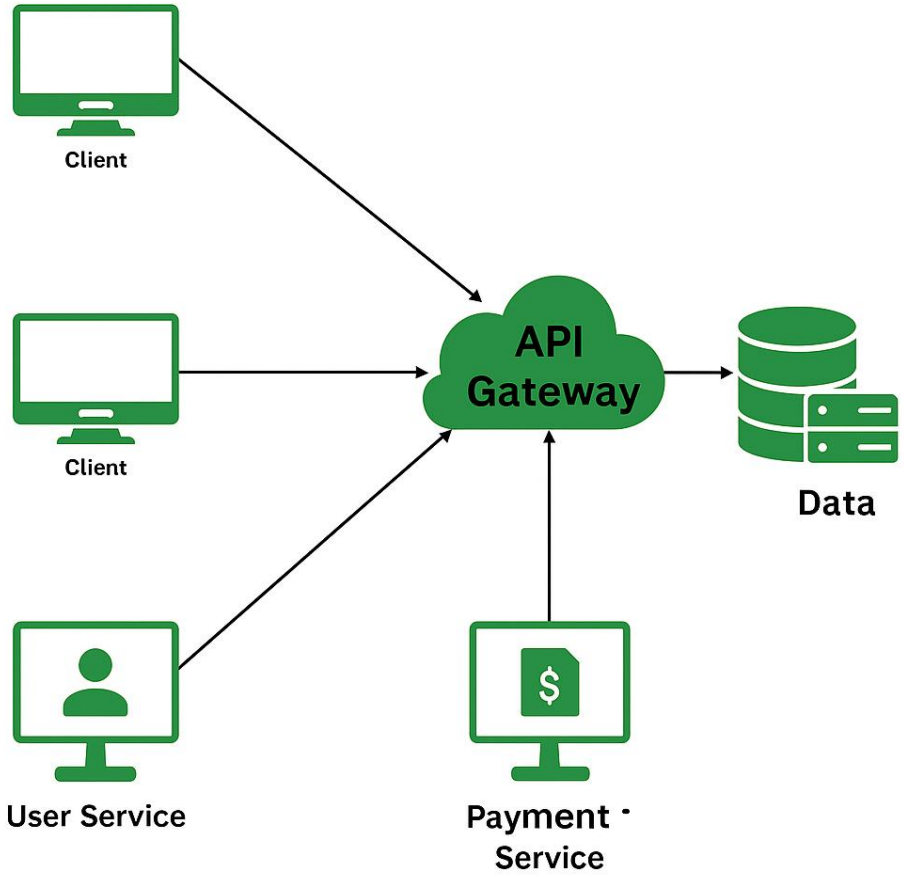


Why would you use this architecture?

Scalability: Each service (User, Payment) can be scaled independently.
Flexibility: Teams can develop, deploy, and update services without impacting others.
Resilience: Failure in one service doesn't necessarily bring down the entire application.

Multi-tier Architecture: The Concept

Three-Tier Architecture

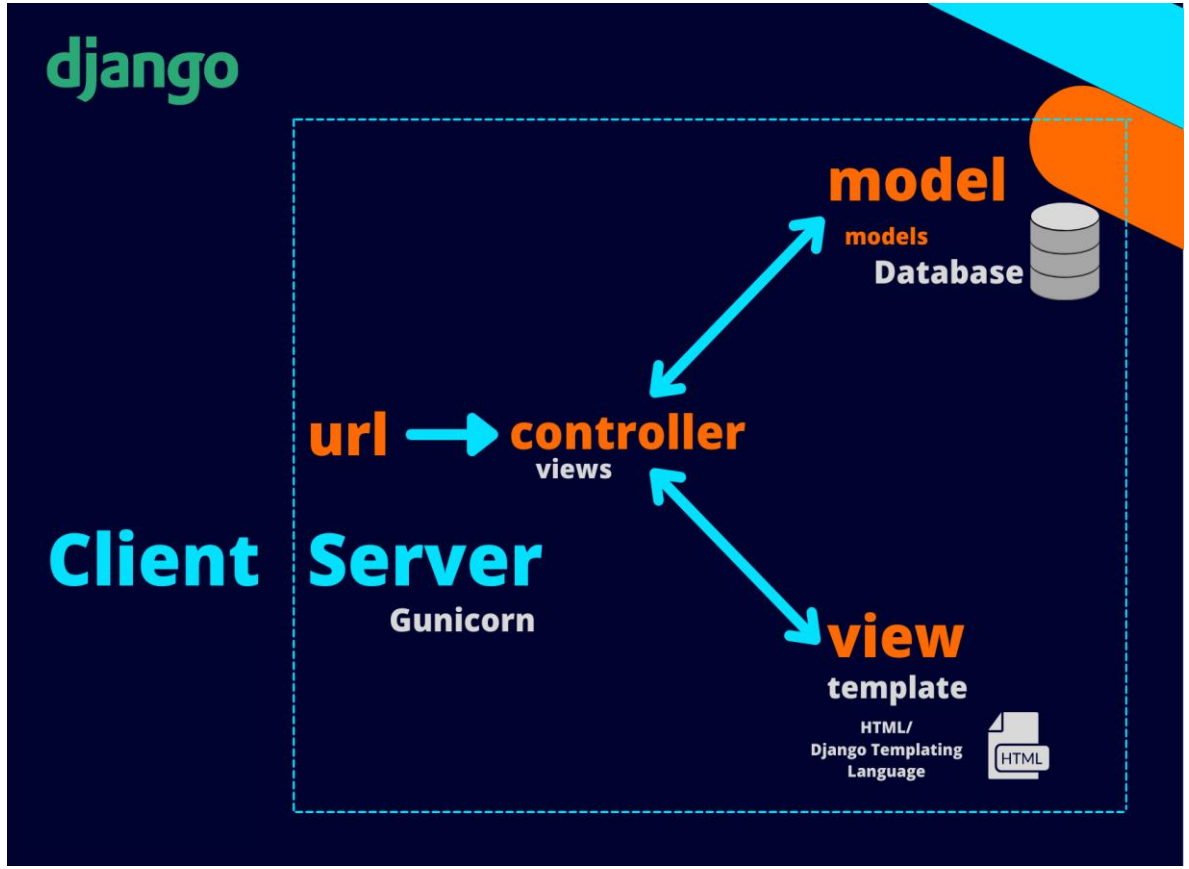


Class Activity:

- ✓ Pick an app you use every day (e.g., Netflix, Uber, Amazon).
- ✓ Name at least three microservices it probably has, and the kind of data each one stores.

Django Framework

Why a framework?



Writing everything from scratch in Python – opening sockets, parsing requests – is error-prone.

Django Philosophy

•*Included*: authentication, admin, ORM, templates, routing.

•*Used by*: Instagram, Pinterest, Mozilla, NASA, etc.

MVT Pattern

•**Model**: Data & rules, stored in database.

•**View**: Python code that receives requests, processes logic, returns response.

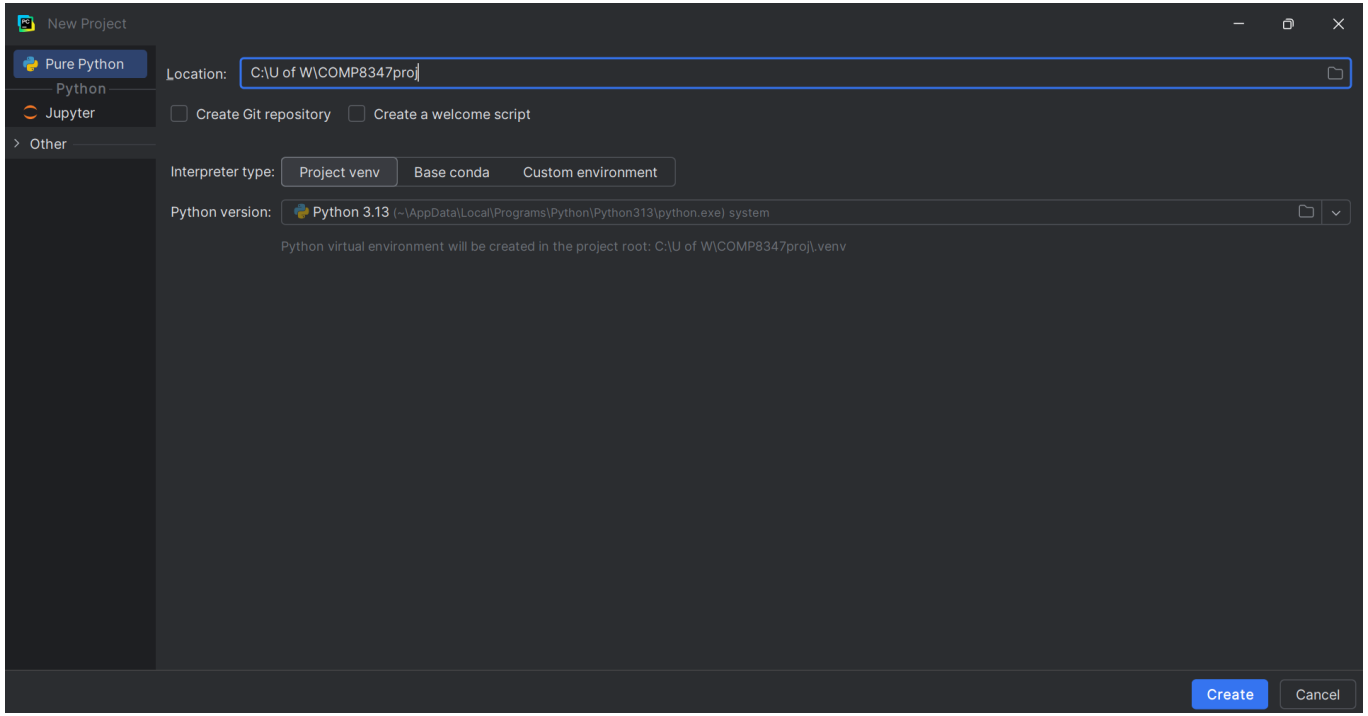
•**Template**: HTML (with Django template language) returned to browser.

Django Framework: Setting Up Django

Step 1: Create a New PyCharm Project

1. Open PyCharm → New Project

2. Location: choose a folder, e.g., `C:\Users\<yourname>\dj_intro` (Windows) or `/Users/<yourname>/dj_intro` (Mac).



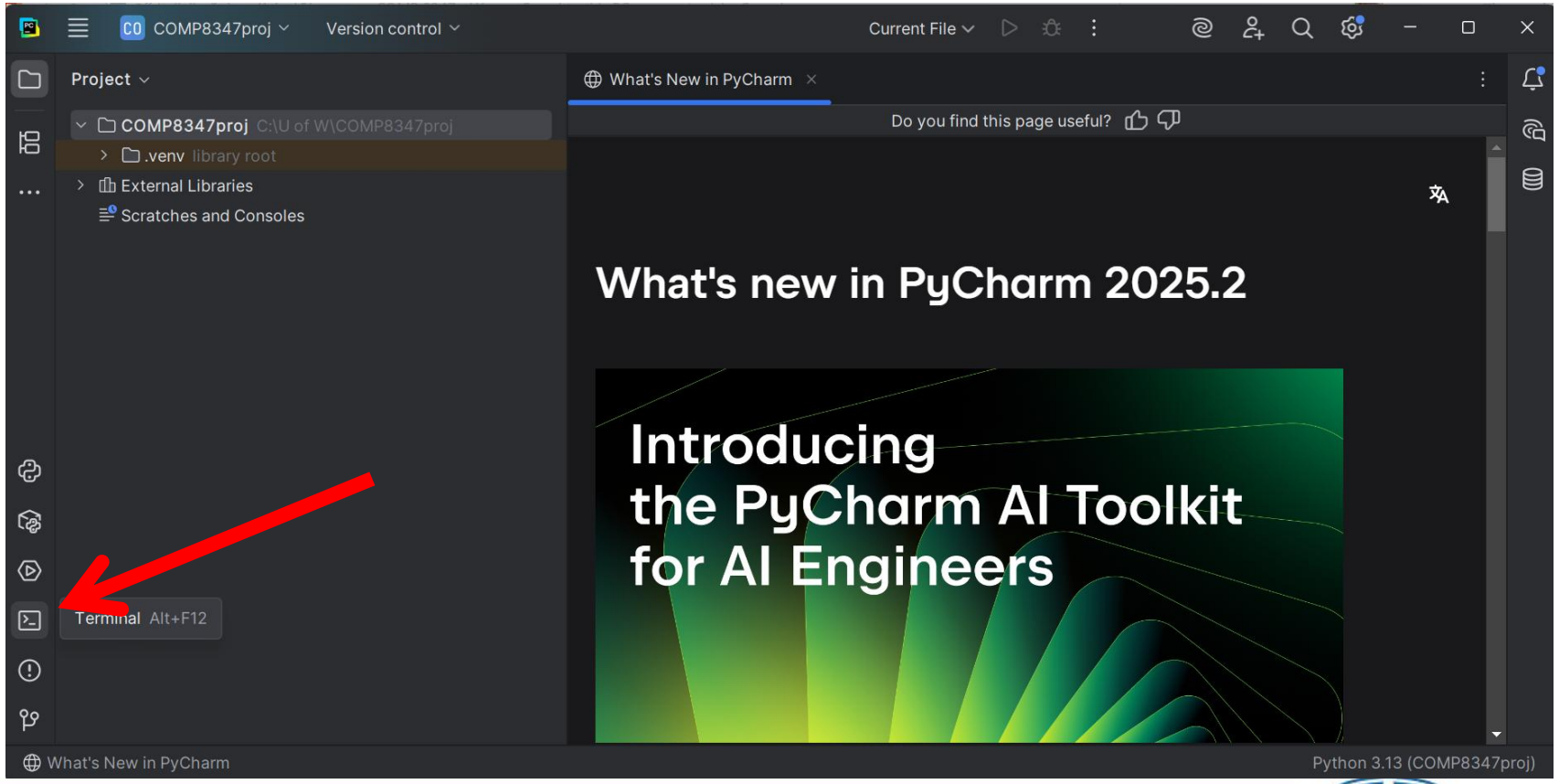
3. Environment: select **Virtualenv** (recommended).

- **Base interpreter**: pick your installed Python 3.x.
- Leave "Inherit global site-packages" **unchecked**.

Django Framework: Setting Up Django

Step 2: Create the Django Project

1. Open PyCharm Terminal



Django Framework: Setting Up Django

Step 3: Install Django

Open PyCharm Terminal (bottom panel) and run:

`pip install django` (Windows)
`pip3 install django` (MacOS)

```
Terminal    Local

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

(.venv) PS C:\U of W\COMP8347proj> pip install django
Collecting django
  Using cached django-5.2.6-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref>=3.8.1 (from django)
  Using cached asgiref-3.9.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.3.1 (from django)
  Using cached sqlparse-0.5.3-py3-none-any.whl.metadata (3.9 kB)
Collecting tzdata (from django)
  Using cached tzdata-2025.2-py2.py3-none-any.whl.metadata (1.4 kB)
Using cached django-5.2.6-py3-none-any.whl (8.3 MB)
Using cached asgiref-3.9.1-py3-none-any.whl (23 kB)
Using cached sqlparse-0.5.3-py3-none-any.whl (44 kB)
Using cached tzdata-2025.2-py2.py3-none-any.whl (347 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.9.1 django-5.2.6 sqlparse-0.5.3 tzdata-2025.2

[notice] A new release of pip is available: 25.1.1 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\U of W\COMP8347proj> pip show django
```

Step 3: Check if Django is installed

Open PyCharm Terminal (bottom panel) and run:

`pip show django`

```
(.venv) PS C:\U of W\COMP8347proj> pip show django
Name: Django
Version: 5.2.6
Summary: A high-level Python web framework that encourages rapid developr
Home-page: https://www.djangoproject.com/
Author:
Author-email: Django Software Foundation <foundation@djangoproject.com>
License: BSD-3-Clause
Location: C:\U of W\COMP8347proj\.venv\Lib\site-packages
Requires: asgiref, sqlparse, tzdata
Required-by:
(.venv) PS C:\U of W\COMP8347proj>
```

Django Framework: Setting Up Django

Step 4: Create the Django Project
Inside the terminal, create the main project:

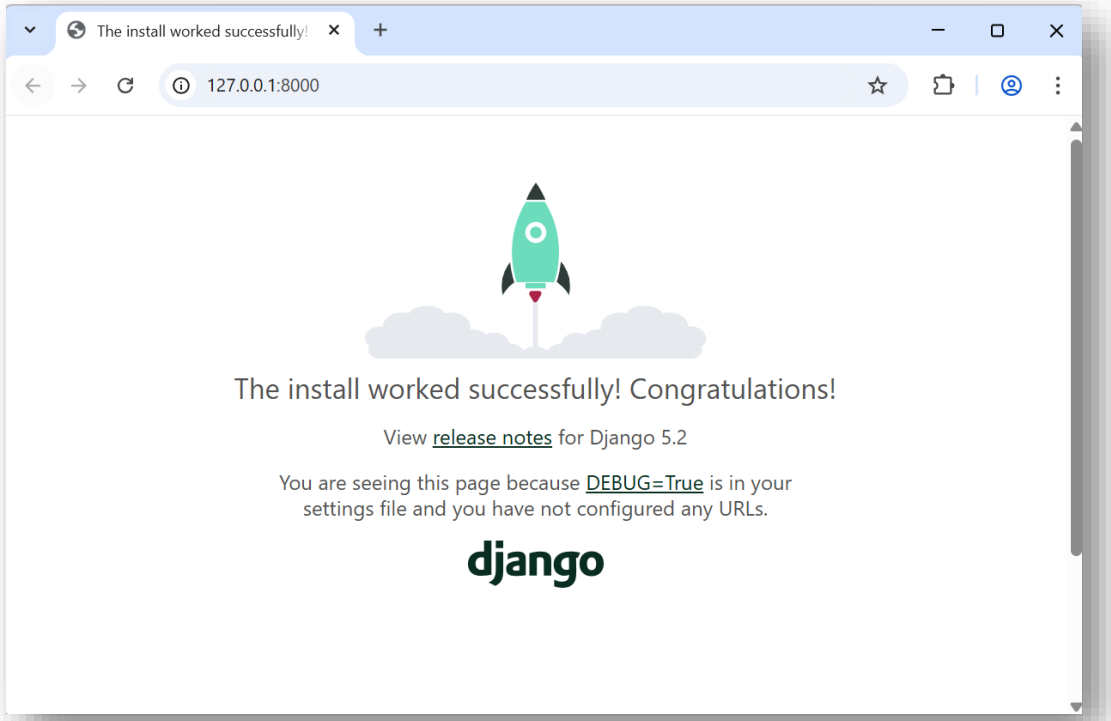
```
django-admin startproject config .
```

(. means "use the current folder.")
PyCharm will detect new files and may ask to add them to Git (optional for now).

Step 5: First Run

```
python manage.py runserver (Windows)
python3 manage.py runserver (MacOS)
```

Open a browser to http://127.0.0.1:8000 – you should see Django's rocket welcome page.

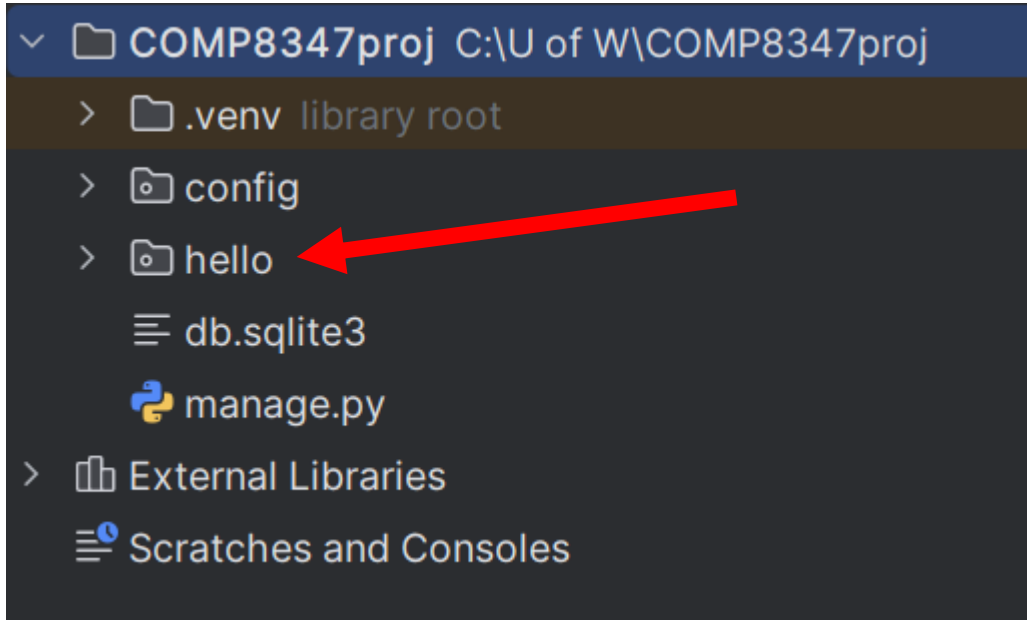


Django Framework: Creating the First App

Step 1: Start the App

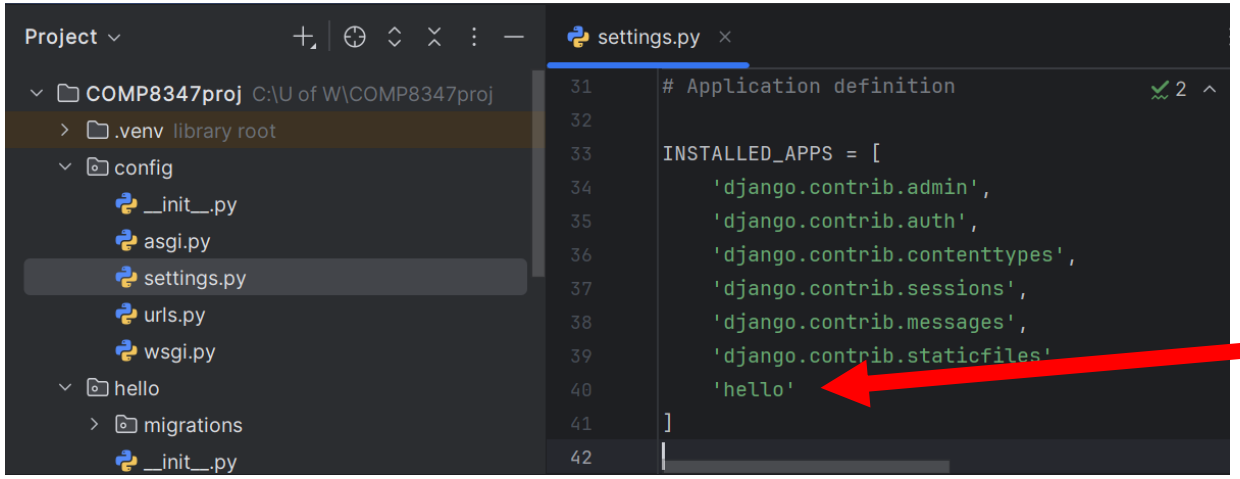
Inside the terminal, run the following:

`python manage.py startapp hello` (Windows)
`python3 manage.py startapp hello` (MacOS)



In PyCharm, you now see a **`hello`** folder with views.py, models.py, etc.

Add **`hello`** to INSTALLED_APPS in **config/settings.py**.



Django Framework: Creating the First App

Step 1: First View

Go to hello/views.py and type the following:

```
from django.http import HttpResponse

def hello_world(request):
    return HttpResponse("Hello, Django World!")
```

Step 2: Add a URL

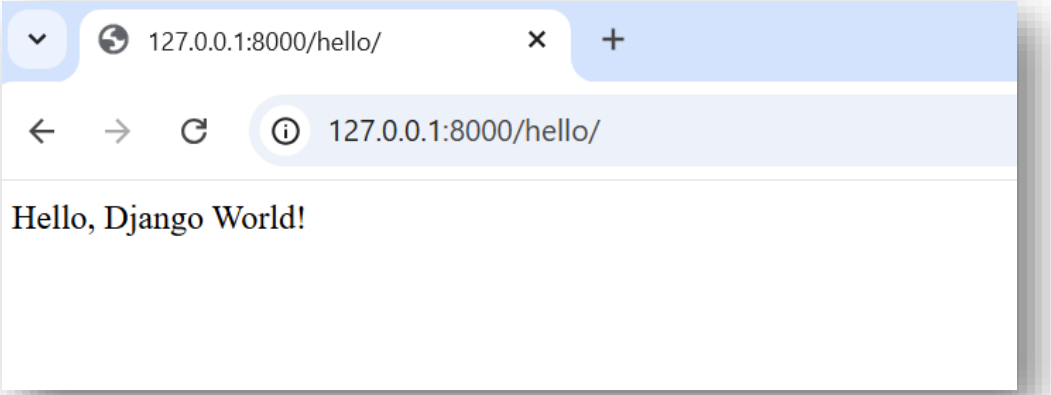
Go to config/urls.py and type the following:

```
from django.urls import path
from hello.views import hello_world

urlpatterns = [
    path("hello/", hello_world, name="hello"),
]
```

Run the server again and visit `http://127.0.0.1:8000/hello/`

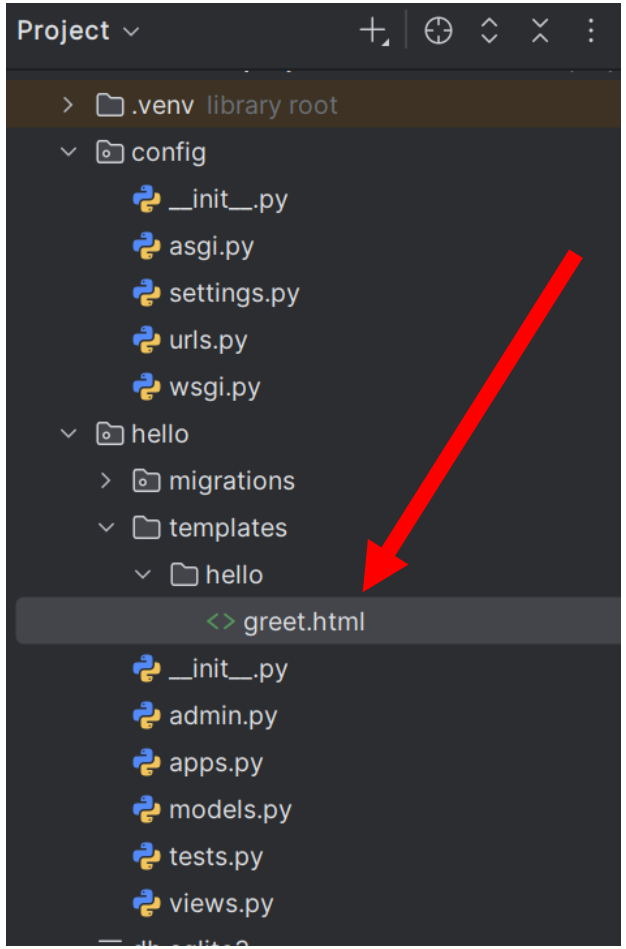
```
python manage.py runserver
```



Django Framework: Creating the First App

Step 3: Template Example

Create `hello/templates/hello/greet.html`

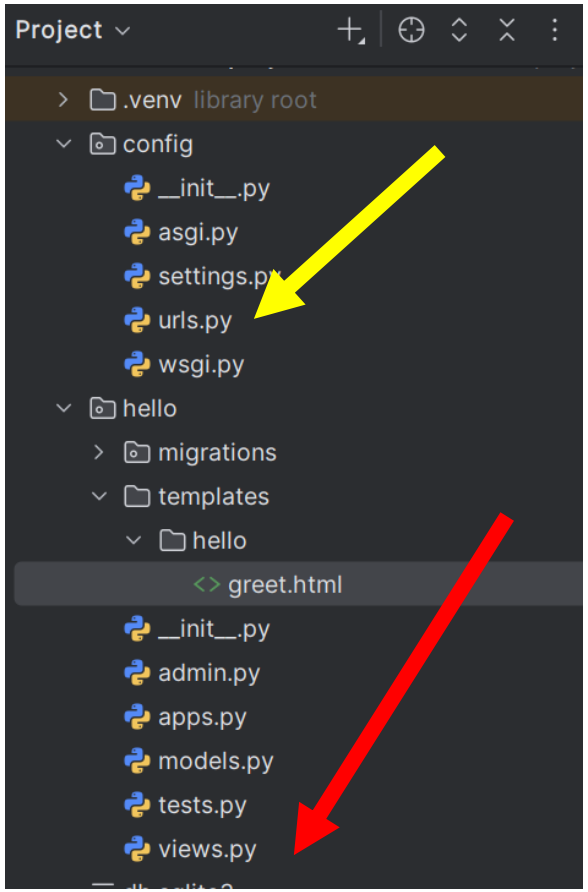



```
<!doctype html>
<html>
  <body>
    <h1>Hello, {{ name }}!</h1>
    <p>This page is rendered using a
Django Template.</p>
  </body>
</html>
```

Django Framework: Creating the First App

Step 3: Template Example

Update `hello/views.py`:



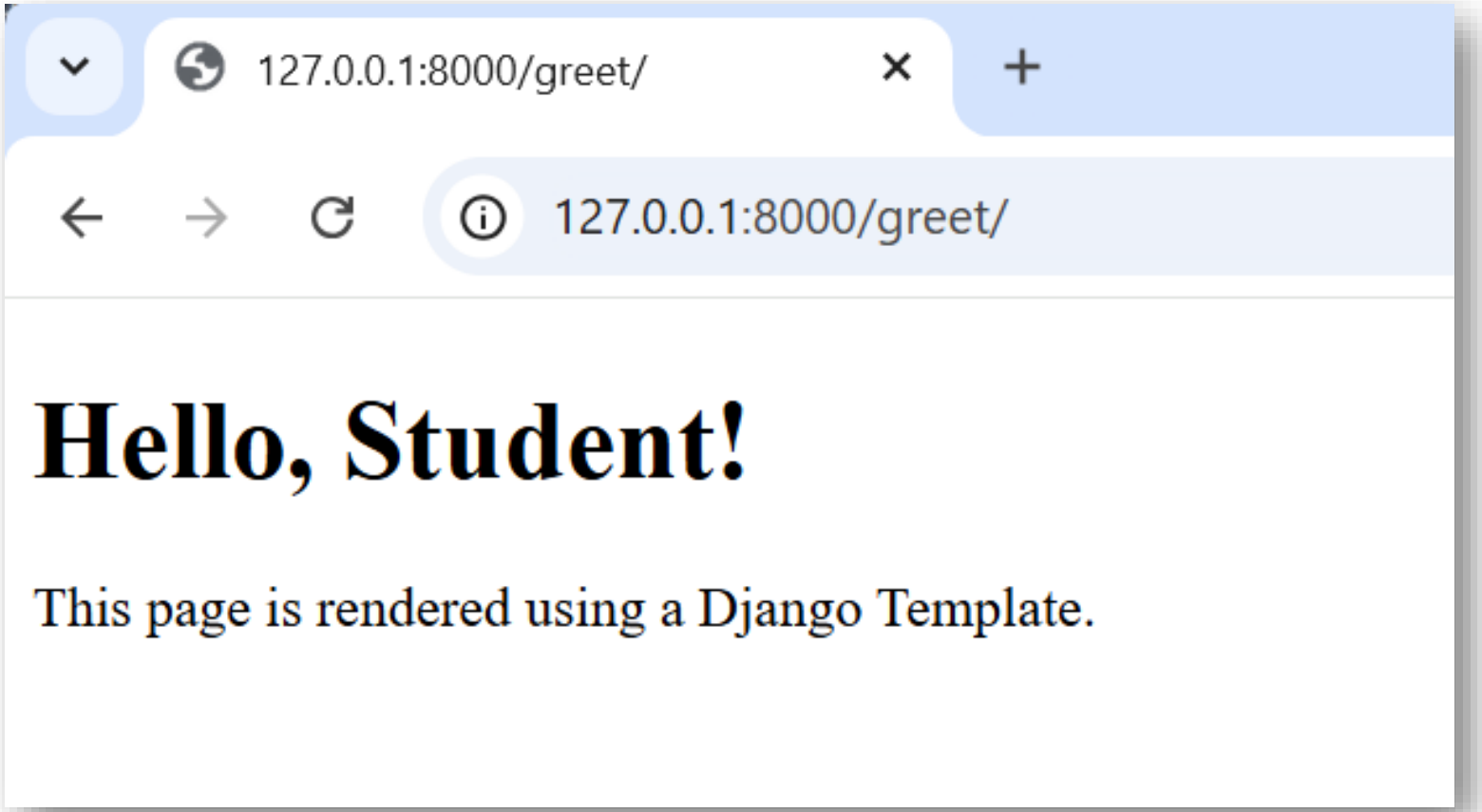

```
from django.shortcuts import render

def greet(request):
    return render(request, "hello/greet.html", {"name": "Student"})
```

Add to `urls.py`:

```
from hello.views import greet
urlpatterns += [path("greet/", greet, name="greet")]
```

Check `http://127.0.0.1:8000/greet/`

Django Framework: Creating the First App

Check `http://127.0.0.1:8000/greet/`